Dynamic Ensemble Strategy for Effective Phishing Detection

UCSC398J

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science

in

Computer Science

by

PRAKASH P

23BCS0097

Under the guidance of

Dr. Chandrasegar T

School of Computer Science Engineering and Information Systems

VIT, Vellore

April, 2026

DECLARATION

I hereby declare that the UCSC398J – Project thesis entitled "**Dynamic Ensemble Strategy for Effective Phishing Detection**" submitted by me, for the award of the degree of ***Bachelor of Science in Computer Science, School of Computer Science Engineering and Information Systems*** to VIT is a record of bonafide work carried out by me under the supervision **Dr. Chandrasegar T, Assistant Professor Sr. Grade 2, SCORE, VIT, Vellore.**

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

PRAKASH P

23BCS0097

CERTIFICATE

This is to certify that the UCSC398J – Project thesis entitled "**Dynamic Ensemble Strategy for Effective Phishing Detection**" submitted by Prakash P (23BCS0097), SCORE, VIT, for the award of the degree of ***Bachelor of Science in Computer Science, School of Computer Science Engineering and Information Systems***, is a record of bonafide work carried out by him under my supervision during the period, 05. 12. 2025 to 17. 04. 2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The thesis fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the Guide

Dr. Chandrasegar T

Assistant Professor Sr. Grade 2, SCORE, VIT, Vellore

Internal Examiner **External Examiner**

Head of the Department

Department of Information Technology

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my **UCSC398J – Project** guide Dr. Chandrasegar T, Assistant Professor Sr. Grade 2, School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore for his constant guidance, continual encouragement, in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of Artificial Intelligence and Educational Technology.

I would like to express my heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Mr. Sankar Viswanathan**, **Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**; and Registrar **Dr. Jayabarathi T**.

My whole-hearted thanks to Dean **Dr. Daphne Lopez,** School of Computer Science Engineering and Information Systems, Head, Department of Smart Computing, **Dr. Vijayan E**; Bachelor of Science in Computer Science Project Coordinator **Dr. Jayaram Reddy**; A, SCORE School Project Coordinator Dr. Thandeeswaran R all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my project successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the project.

Place: Vellore

Date: **Prakash P**

Executive Summary

Phishing attacks are among the most common and dangerous cybersecurity threats today. These attacks steal sensitive information such as usernames, passwords, and banking details by deceiving users into visiting malicious websites that appear legitimate. This project presents an intelligent phishing detection system using a deep learning approach. A Dynamic Ensemble Strategy utilizing Deep Neural Networks (DNN) is developed to analyze URL structures and domain features for accurate classification. Unlike traditional blacklist methods, the model can detect both known and unseen phishing patterns, making it adaptive to evolving cyber threats.

The system functions as a web-based application where users input URLs and receive real-time predictions, along with a screenshot of the webpage for verification. The backend features a trained DNN model, while the frontend provides an interactive interface hosted on a cloud platform. The Flask framework is used for backend processing, ensuring scalability and efficient deployment. This architecture enables immediate responses without reliance on manual verification or static filtering.

The key outcome includes a highly accurate automated phishing detection tool that reduces false positives and enhances reliability, ensuring faster decision-making and improved user trust. It also demonstrates real-time deployment capability and seamless integration of deep learning with web technology. The model is optimized for real-time performance, providing a robust solution for modern phishing threats.

CONTENTS

TITLE	Page No.
Acknowledgement	**iv**
Executive summary	**v**
Table of contents	**vi**
List of figures	**viii**
List of tables	**ix**
Abbreviations	**x**
Symbols and notations	**xi**
Chapter 1: Introduction	**1**
1.1 Objective	2
1.2 Motivation	3
1.3 Background	4
Chapter 2: Description and goals	**5**
2.1 Problem statement	6
2.2 Scope of the project	7
2.3 Gap analysis	9
2.4 Proposed system	11
Chapter 3: Technical specification	**12**
3.1 Software requirements	12
3.2 Functional requirements	13
3.3 Non-functional requirements	14
3.4 Hardware and system requirements	15
Chapter 4: Design approach and details	**17**
4.1 Literature survey	17
4.2 Findings from literature survey	18
4.3 System architecture	20

4.3.1 User interface 22
4.3.2 Request handler 22
4.3.3 URL analysis module 23
4.3.4 Screenshot module 23
4.3.5 Domain intelligence module 24
4.3.6 Feature engineering 24
4.3.7 Deep DNN model 25
4.3.8 Output & dashboard 25
4.4 UML diagrams 26

Chapter 5: Schedule, tasks and milestones 28

5.1 Project timeline 28
5.2 Phase-wise description 29

Chapter 6: Dissertation demonstration 34

6.1 Implementation overview 34
6.2 Sample code 36
6.2.1 Feature Extraction 36
6.2.2 Deep Neural Network 37
6.2.3 Model Training 38
6.2.4 Prediction Function 38
6.2.5 Flask Backend API 38
6.2.5 Performance Metrics 39
6.3 System screenshots 40

Chapter 7: Result and discussion 42

7.1 Model performance evaluation 42
7.2 System usability testing 43
7.3 Cost analysis 43
7.4 Discussion 44

Chapter 8: Summary 45

8.1 Project Overview 45
8.2 Key Achievements 46
8.3 Conclusion 46

Chapter 9: References **47**

Appendix A 49

List of Figures

Figure No.	Title	Page No.
4.1	Phishing Detection System Architecture Diagram	21
4.2	Use CasDiagrams - Phishing Detection	27
6.1	Index Page	40
6.2	User Input Page	40
6.3	Loading Screen After the URL is Submitted	41
6.4	Prediction Output	41
6.5	Recent Scan Result	41

List of Tables

Table No.	Title	Page No.
3.1	Software Requirements	13
3.2	Functional Requirements Specification	14
3.3	Non-Functional Requirements	15
4.1	Literature Survey	17
5.1	Project Schedule and Milestones	28
7.1	Model Performance Metrics	42
7.2	Project Cost Analysis	43

List of Abbreviations

Abbreviation	Expanded Form
URL	Uniform Resource Locator
API	Application Programming Interface
IP	Internet Protocol
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
SSL	Secure Sockets Layer
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
JSON	JavaScript Object Notation
CSV	Comma Separated Values
TF	TensorFlow
DNN	Deep Neural Network
ReLU	Rectified Linear Unit
FN	False Negative
FP	False Positive
TP	True Positive
TN	True Negative

Symbols and Notations

Symbol / Notation	Description
X	Input feature matrix for the model
y	Target labels (1 = phishing, 0 = legitimate)
X_{train}, X_{test}	Training and testing feature sets
y_{train}, y_{test}	Training and testing label sets
μ	Mean used in normalization
σ	Standard deviation for scaling
$Z = \frac{X - \mu}{\sigma}$	Feature standardization formula
$f(x)$	Neural network mapping function
W	Weights of the model
b	Bias term
$z = Wx + b$	Linear combination in a neuron
$p > 0.5$	Threshold for classification
$\text{ReLU}(x)$	Non-linear activation function
$\sigma(x)$	Sigmoid activation for output
$\hat{y}$	Predicted output
θ	Model parameters (weights & bias)
α	Learning rate
TP, TN, FP, FN	Confusion matrix components
Accuracy	Overall prediction correctness
Precision	Correct positive predictions
Recall	Detection of actual positives

CHAPTER 1

INTRODUCTION

Phishing attacks are one of the most serious cybersecurity threats in the modern digital world. Phishing is a type of cybercrime in which attackers create fake websites, emails, or messages that appear to originate from trusted and legitimate sources in order to deceive users. The primary objective of these attacks is to steal sensitive information such as usernames, passwords, banking credentials, credit card details, and personal identification data. Once users unknowingly enter their information on these fraudulent platforms, attackers exploit it for financial fraud, identity theft, and unauthorized access to personal accounts.

With the rapid growth of online services such as internet banking, digital payments, e-commerce platforms, and social media applications, phishing attacks have become increasingly common and dangerous. Modern phishing attacks are highly sophisticated and are designed to closely resemble legitimate websites, making them difficult to identify through visual inspection alone. Attackers continuously update their strategies to bypass existing security mechanisms and exploit human trust.

Phishing websites often use deceptive techniques such as similar-looking domain names, misleading URL structures, shortened links, hidden redirections, and fake security indicators to trick users into believing they are legitimate. These attacks primarily target human behavior rather than technical vulnerabilities, which increases their effectiveness. Many users are unaware of these manipulations, leading to a higher success rate of phishing attempts.

To address this issue, this project focuses on developing a deep learning-based phishing detection system that analyzes and classifies URLs based on their structural and domain-related features. The system identifies patterns such as abnormal URL length, presence of suspicious keywords, HTTPS inconsistencies, and domain age variations to determine whether a website is safe or malicious. It is implemented as a web-based application where users can input URLs and receive real-time predictions through a simple and user-friendly interface in real-world environments.

1.1 Objective

The objective of this project is to develop a phishing detection system using a deep learning approach. The system classifies URLs as legitimate or phishing based on important features. It focuses on improving accuracy, speed, and real-time detection capability.

The specific objectives of the project are as follows:

- A deep learning model is developed for phishing URL classification. The model learns patterns from URL structure and domain-related information, improving detection accuracy compared to traditional methods.
- URL structure and domain features are analyzed to identify phishing websites. Attributes such as URL length, suspicious keywords, special characters, and domain properties are considered to detect malicious patterns effectively.
- A real-time web application is implemented for user interaction. Users can input URLs and receive instant prediction results through a simple and user-friendly interface.
- The system provides fast and accurate URL prediction using a trained model. It processes input efficiently in real time to ensure quick and reliable results.
- The system enhances cybersecurity by effectively detecting phishing websites. It helps users avoid malicious platforms and reduces the risks of data theft and financial fraud.

1.2 Motivation

Phishing attacks have become one of the most common and dangerous cyber threats in recent years. Attackers use fake websites and deceptive URLs to steal sensitive information such as login credentials, personal data, and banking details. With the increasing use of online services, many users unknowingly fall victim to such attacks due to their realistic appearance and convincing design.

One of the major reasons behind the success of phishing attacks is the lack of user awareness and the inability to distinguish between legitimate and fraudulent websites. Attackers exploit this weakness by creating websites that closely resemble trusted platforms. As a result, users often trust these websites and provide their confidential information without realizing the risk involved.

Traditional phishing detection systems are often unable to identify newly generated phishing URLs, as they rely on predefined rules or blacklists. This limitation makes them ineffective against modern phishing techniques, where attackers continuously modify their strategies to bypass detection. Hence, there is a strong need for a more intelligent and adaptive solution that can handle dynamic threats.

The motivation behind this project is to develop an automated phishing detection system using deep learning techniques that can analyze URLs in real time and classify them accurately. Deep learning models have the ability to learn complex patterns and detect even unseen phishing URLs, making them more effective than traditional methods.

Furthermore, this project aims to reduce the impact of phishing attacks by improving detection accuracy and minimizing false predictions. It also contributes to increasing awareness about cybersecurity threats and encourages users to adopt safe online practices. Overall, the motivation is to build a reliable, efficient, and practical solution that enhances digital security and protects users from phishing attacks.

1.3 Background

Phishing detection has been an active area of research in cybersecurity due to the increasing number of online threats. Early detection systems were primarily based on blacklist approaches, where known phishing URLs were stored in databases and compared against user input. While effective for detecting previously identified threats, these systems fail to recognize newly created phishing websites, limiting their effectiveness.

To overcome these limitations, machine learning techniques were introduced, which analyze features of URLs and classify them based on predefined patterns. These methods improved detection accuracy but still required manual feature selection and were less effective in identifying complex patterns in data.

With advancements in artificial intelligence, deep learning approaches have emerged as a powerful solution for phishing detection. Deep learning models, such as Deep Neural Networks (DNN), can automatically learn complex relationships and hidden patterns from large datasets without requiring extensive manual feature engineering. This makes them highly effective in detecting both known and unknown phishing URLs.

In this project, a deep learning-based approach is used to enhance phishing detection by analyzing structural and domain-based features of URLs. The system integrates these techniques into a web-based application, providing real-time detection and improving user security. This background highlights the evolution of phishing detection methods and justifies the need for an advanced, intelligent solution.

CHAPTER 2

DESCRIPTION AND GOALS

Online security has become a critical concern due to the rapid rise of phishing attacks that target users through fake websites and deceptive URLs. These attacks are designed to steal sensitive information such as usernames, passwords, and banking credentials, making them a serious cybersecurity threat. In recent years, the frequency and complexity of phishing attacks have increased significantly, affecting both individuals and organizations. Attackers are continuously developing new techniques to make phishing websites appear more realistic and trustworthy, which increases the difficulty of detection. As a result, users are often unable to identify these threats, leading to serious consequences such as financial loss and data breaches.

This project addresses this issue by developing a deep learning-based phishing detection system that analyzes URL patterns and domain features to classify websites as legitimate or phishing. The system focuses on improving detection accuracy and adapting to evolving attack techniques. By leveraging deep learning algorithms, the system is capable of learning complex patterns from large datasets, enabling it to detect both known and unknown phishing URLs effectively. This makes it more reliable compared to traditional detection methods that rely on predefined rules or blacklists.

A web-based application is developed as part of this project to provide real-time phishing detection services. Users can enter URLs and instantly receive classification results. The backend uses a trained deep neural network model, while the frontend offers a simple and interactive interface for easy use. The system ensures fast processing of input data and delivers immediate results. It is also designed with scalability in mind, allowing future improvements with updated datasets and enhanced model architectures. The integration of backend and frontend components ensures smooth communication and efficient performance of the system.

2.1 Problem Statement

Phishing attacks have become increasingly common and sophisticated, making it difficult for users to identify malicious websites. Attackers create deceptive URLs and fake web pages that closely resemble legitimate platforms to trick users into providing sensitive information such as usernames, passwords, and banking details. These fraudulent websites often mimic the design, layout, and branding of well-known organizations, making them appear authentic to unsuspecting users.

One of the major challenges in phishing detection is that users are often unable to distinguish between genuine and fraudulent websites due to their similar appearance. Attackers use techniques such as domain spoofing, URL manipulation, and social engineering to make phishing websites more convincing. This leads to serious consequences such as data theft, financial loss, identity theft, and privacy breaches.

Another critical issue is the limitation of existing detection systems. Traditional approaches, such as blacklist-based methods, can only detect known phishing URLs that have already been identified. However, attackers continuously generate new phishing websites with slight variations in URLs, making it difficult for these systems to keep up. This results in many phishing attacks going undetected.

Furthermore, rule-based and traditional machine learning approaches often fail to capture complex patterns in phishing URLs. These methods require manual feature selection and are not flexible enough to adapt to evolving attack strategies. As phishing techniques continue to advance, there is a need for more intelligent and adaptive detection systems.

Therefore, there is a need for an automated solution that can analyze URL patterns and domain features to accurately classify websites in real time. Such a system should be capable of detecting both known and unknown phishing URLs while providing fast and reliable results. This project aims to address these challenges by developing a deep learning-based phishing detection system that improves detection accuracy and enhances cybersecurity protection.

2.2 Scope of the Project

This project focuses on developing an intelligent system to detect phishing websites using deep learning techniques. It aims to provide real-time analysis of URLs and help users identify malicious links effectively. The system is designed to enhance cybersecurity by offering a reliable and automated solution for phishing detection. The scope of the project covers both the technical implementation and practical usability of the system.

The system primarily focuses on URL-based phishing detection, where input URLs are analyzed using various structural and domain-related features. These features include URL length, presence of special characters, suspicious keywords, domain age, and HTTPS usage. By analyzing these attributes, the system can identify patterns that indicate whether a website is legitimate or malicious. This approach ensures accurate classification and improves detection performance.

Another important aspect of the project scope is real-time processing. The system is designed to provide instant classification results when a user inputs a URL. This is achieved through efficient backend processing and optimized model performance. Real-time detection is essential in practical scenarios, as users need immediate feedback before accessing a potentially harmful website. The system ensures minimal delay while maintaining high accuracy.

The project also includes the development of a web-based interface that allows users to interact with the system easily. The interface is designed to be simple, intuitive, and user-friendly, ensuring accessibility for users with different levels of technical knowledge. Users can enter URLs and view results without requiring any specialized skills. This enhances the usability and effectiveness of the system.

In addition to classification, the system provides a screenshot feature for visual verification of the entered URL. This helps users understand how the website appears and identify any suspicious elements visually. This feature adds an extra layer of security and improves user confidence in the system's predictions.

The scope also includes scalability and future enhancements. The system is designed to handle larger datasets and increased user traffic. The model can be retrained with updated datasets to improve performance and adapt to new phishing techniques. Additional features, such as browser integration and advanced analytics, can be incorporated in future versions of the system.

The system scope includes the following key aspects:

- The system focuses on detecting phishing websites using deep learning techniques.
- It performs real-time classification of URLs as legitimate or phishing.
- The model analyzes URL structure and domain-based features for prediction.
- It provides a web-based interface for users to check suspicious URLs easily.
- The system helps in reducing risks of credential theft and online fraud.
- It can be extended to integrate with browsers and security tools in future.
- The model is scalable and can be enhanced with new datasets and features.
- The system supports continuous improvement through model retraining.
- It provides visual verification using webpage screenshots.

Overall, the project scope is focused on delivering a practical, scalable, and efficient phishing detection solution that can be used in real-world applications.

2.3 Gap Analysis

The rapid increase in phishing attacks has exposed several limitations in existing detection systems. Traditional approaches rely heavily on blacklist-based methods, which store known phishing URLs and compare them against user input. While effective for detecting previously identified threats, these systems fail to detect newly generated phishing URLs. Attackers continuously create new and unique URLs with slight variations, making it difficult for blacklist systems to keep up. This creates a significant gap in cybersecurity, as many phishing websites remain undetected and continue to target users.

Another major limitation of existing systems is low detection accuracy. Rule-based approaches and basic machine learning models often rely on predefined conditions and limited feature sets. These models are not capable of capturing complex relationships and hidden patterns present in phishing URLs. As a result, they may fail to correctly classify sophisticated phishing attacks, especially those designed to bypass traditional detection techniques. This limitation reduces the overall effectiveness of such systems in real-world scenarios.

High false positive rates are also a common issue in existing detection systems. Many models incorrectly classify legitimate websites as phishing due to similarities in certain features. This can cause inconvenience to users and reduce trust in the system. In critical applications, false positives can lead to unnecessary restrictions and impact user experience. Therefore, reducing false predictions is an important requirement for improving system reliability.

Lack of real-time detection is another gap observed in traditional systems. Many existing tools require manual verification or depend on periodic updates, which leads to delayed results. In practical situations, users need immediate feedback to decide whether a website is safe or not. Delayed detection increases the risk of users interacting with malicious websites before receiving any warning.

Another gap is limited user interaction and usability. Many existing tools do not provide simple and intuitive interfaces for users. This makes it difficult for non-technical users to utilize these systems effectively. A lack of user-friendly design reduces the practical applicability of phishing detection tools and limits their adoption.

In addition, most systems do not provide visual verification of websites. Users are often given only a classification result without any supporting information. Without a preview or screenshot, it becomes difficult for users to understand the nature of the detected threat. This lack of transparency affects user confidence and decision-making.

Scalability is another concern in existing systems. Many traditional approaches are not designed to handle large volumes of data or increased user requests. As the number of internet users grows, detection systems must be capable of handling high traffic efficiently without affecting performance. Limited scalability restricts the usability of such systems in real-world applications.

To address these gaps, this project proposes a deep learning-based phishing detection system that overcomes the limitations of traditional approaches. The system uses a Deep Neural Network (DNN) model to improve detection accuracy by learning complex patterns from data. It reduces false positives by optimizing model performance and ensures real-time detection through efficient backend processing.

The proposed system also improves adaptability by allowing the model to be retrained with updated datasets, enabling it to detect new phishing techniques. A user-friendly web interface is provided to enhance usability, making the system accessible to all users. Additionally, the inclusion of a screenshot feature provides visual verification, improving user confidence in the results.

Overall, the proposed solution effectively bridges the gaps identified in existing systems by providing an accurate, scalable, and real-time phishing detection framework.

2.4 Proposed System

The proposed system focuses on developing an intelligent phishing detection framework using a deep learning-based approach. The system is designed to analyze URLs and classify them as legitimate or phishing by extracting important structural and domain-related features. A Deep Neural Network (DNN) model is used to learn complex patterns from the dataset, enabling accurate identification of both known and newly generated phishing websites. This approach improves detection capability compared to traditional methods and provides better adaptability to evolving cyber threats.

The system is implemented as a web-based application that allows users to input suspicious URLs and receive real-time predictions. The backend integrates the trained deep learning model, which processes the input URL and generates classification results instantly. In addition to prediction, the system also captures a screenshot of the entered webpage to provide visual verification for the user. The frontend is designed to be simple and interactive, ensuring ease of use for users with minimal technical knowledge.

The use of the Flask framework enables efficient communication between the user interface and the backend model. The system ensures fast processing, high accuracy, and reliable performance. It eliminates the dependency on manual verification and static blacklist methods by providing automated detection.

The proposed system offers several advantages, including high accuracy, reduced false positives, and fast response time. It is scalable and can be enhanced with additional features and updated datasets in the future. Overall, the proposed solution provides an effective and reliable approach to detect phishing websites and improve cybersecurity protection in real-world environments.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 Software Requirements

The software requirements for this phishing detection system focus on supporting deep learning-based URL classification and real-time web application functionality. Python is used for implementing the model and backend processing, while Flask manages server-side communication. Deep learning libraries are used to load and execute the trained model for prediction. Data processing tools handle feature extraction from URLs, and web technologies provide a simple interface for user interaction. These components ensure accurate and efficient phishing detection.

URL Feature Extraction and Processing

- ✓ URL length, keywords, and structural features are extracted for analysis.
- ✓ Domain-related attributes are analyzed for classification.
- ✓ Input URLs are converted into model-compatible format.
- ✓ Real-time user input is processed for feature extraction.
- ✓ Feature processing is integrated with backend module.

Model Execution and Prediction

- ✓ Trained deep neural network model is loaded for prediction.
- ✓ URLs are classified as legitimate or phishing in real time.
- ✓ Input data is processed efficiently using trained model weights.
- ✓ Prediction results are generated for user display.
- ✓ Unseen phishing patterns are detected using learned features.

Table 3.1: Software requirements

Component	Technology	Purpose
Programming Language	Python	Used to implement backend logic and phishing detection model
Backend Framework	Flask	Handles server-side processing and API communication
Deep Learning Model	TensorFlow / Keras	Builds and runs the DNN model for URL classification
Data Processing	NumPy / Pandas	Processes data and extracts URL-based features
Feature Extraction	Custom Python Scripts	Extracts structural and domain-related URL features
Frontend Interface	HTML, CSS, Bootstrap	Provides user interface for URL input and result display
Deployment Platform	Hugging Face	Hosts the web application for online access
Version Control	Hugging Face Spaces	Manages code updates and version tracking within the platform

3.2 Functional Requirements

The functional requirements define the core operations of the phishing detection system. These requirements describe how the system interacts with users and processes data to deliver accurate results. The system is designed to provide a seamless experience where users can easily input URLs and receive instant predictions.

The system accepts user input in the form of a URL through the web interface. Once the input is received, the backend processes the URL by extracting relevant features such as length, structure, and domain attributes. These features are then passed to the trained deep learning model, which analyzes the input and classifies it as legitimate or phishing.

The prediction result is displayed to the user in real time through the interface. In addition to classification, the system also captures a screenshot of the entered webpage to provide visual verification. This helps users better understand the nature of the website and improves decision-making.

The system ensures efficient data flow between components, enabling fast processing and accurate predictions. Each function is designed to operate seamlessly, ensuring reliability and usability.

Table 3.2: Functional Requirements Specification

S.No	Function	Description
1	URL Input	Accepts URL entered by the user through the web interface
2	Model Prediction	Classifies URL using trained deep learning model
3	Result Display	Shows prediction result to the user in real time
4	Screenshot Capture	Captures webpage screenshot for visual verification
5	Real-time Processing	Processes input and provides instant output

3.3 Non-Functional Requirements

The non-functional requirements define the performance, usability, and reliability aspects of the phishing detection system. These requirements ensure that the system operates efficiently and provides a good user experience.

The system is designed to provide fast response time, enabling real-time URL classification without delay. High accuracy is maintained to ensure reliable detection of phishing websites while minimizing false predictions. Reliability is an important factor, as the system must perform consistently without failure.

Usability is also a key consideration. The interface is designed to be simple and easy to use, allowing users to interact with the system without technical difficulty. Scalability ensures that the system can handle increased user traffic and larger datasets without affecting performance.

Table 3.3: Non-Functional Requirements

S.No	Requirement	Description
1	Performance	Provides fast response for real-time URL classification
2	Accuracy	Ensures high prediction accuracy with minimal false positives
3	Reliability	Performs consistently without failure
4	Usability	Provides simple and user-friendly interface
5	Scalability	Handles increased number of users and data efficiently

3.4 Hardware and System Requirements

The hardware and system requirements define the resources needed to develop, train, and deploy the phishing detection system. These requirements are categorized based on end users and development environments to ensure flexibility and accessibility.

For End Users:

- ✓ Computer, laptop, or smartphone for accessing the system
- ✓ Internet connection for using the web-based application
- ✓ Modern web browser (Chrome, Edge, Firefox) for interaction
- ✓ Display device for viewing prediction results and screenshots

For Development and Training:

- ✓ Processor: Intel Core i5 / i7 or equivalent for efficient processing
- ✓ RAM: Minimum 4 GB (8 GB recommended for better performance)
- ✓ GPU: NVIDIA GPU or cloud-based GPU (e.g., Google Colab) for faster model training
- ✓ Storage: At least 10–20 GB free space for datasets and model files

Software Requirements:

- ✓ Operating System: Windows / Linux / macOS
- ✓ Programming Language: Python, Flask
- ✓ Frameworks: TensorFlow / Keras
- ✓ Libraries: NumPy, Pandas, Matplotlib
- ✓ Development Tools: Jupyter Notebook / Google Colab / VS Code

Deployment Requirements:

- ✓ The trained model can be deployed as a web application
- ✓ The system supports cloud-based deployment for accessibility
- ✓ Can be integrated with browsers or security tools in future
- ✓ Optimized for efficient performance using lightweight models

System Efficiency

The system is designed to ensure optimal performance using efficient deep learning techniques:

- ✓ Fast prediction time for real-time classification
- ✓ Reduced computational cost for better performance
- ✓ Ability to run on standard devices without high-end hardware
- ✓ Scalable architecture to handle increased user requests

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 Literature Survey

A literature survey was conducted to study phishing detection using machine learning and deep learning techniques. The following table presents the ten key research papers reviewed, along with their merits and limitations:

Table 4.1: Literature Survey

S.No	Title	Author & Year	Merits	Demerits
1	Hybrid Deep Learning Ensemble for Real-Time Phishing Detection	Prabavathi & MathanKumar (2025)	Achieves high accuracy (99.2%) with low latency and reduced feature complexity. Suitable for real-time detection	High computational complexity and requires large datasets for effective performance
2	Siamese LSTM-Based Framework for Phishing URL Detection	Sruthi & Manohar Naik (2025)	Works well with limited data and adapts to new phishing patterns using similarity learning	Higher computational cost due to pairwise comparisons during prediction
3	Deep Learning-Based Phishing Detection using PhiUSIIL Dataset	Rawla et al. (2025)	Provides high precision using updated datasets and improves detection accuracy	Lacks explainability and requires high computational resources
4	BiLSTM with Attention for URL-Based Phishing Detection	Akçam et al. (2025)	Captures contextual patterns effectively and works across different languages	Slow training process and high memory usage
5	Feature Selection-Based Machine Learning for Phishing Detection	Nayak et al. (2025)	Achieves good accuracy with fewer features, reducing computational cost	May fail to capture complex relationships in URL patterns

6	Fuzzy Deep Neural Stacked Autoencoder for Phishing Detection	Vidyasri & Suresh (2025)	Automatically learns features and handles noisy data efficiently	High memory usage and complex model structure
7	Ensemble Deep Learning Framework for Phishing Detection and Extraction	Prasad & Dondeti (2025)	Handles imbalanced data and provides fast and accurate detection	Depends on external sources, which may increase latency
8	PhishHunter-XLD: Ensemble Model for Phishing URL Classification	Doshi et al. (2025)	Provides very high accuracy and captures contextual meaning effectively	Requires high computational power and complex architecture
9	Attention-Based Deep Learning for URL Feature Phishing Detection	Nguyen & Huong (2025)	Achieves high accuracy with balanced performance using attention mechanisms	Performance depends heavily on dataset quality
10	Real-Time Phishing Detection using Character-Level CNN	Avila-Gutierrez et al. (2025)	Enables real-time detection without external data sources	Limited ability to capture deep contextual information

4.2 Findings from Literature Survey

The literature survey conducted on various phishing detection techniques highlights that deep learning and ensemble-based approaches provide highly accurate and efficient solutions. From the reviewed studies, it is observed that hybrid and ensemble models, such as the work by Prabavathi & MathanKumar (2025) and Doshi et al. (2025), achieve very high accuracy levels, often exceeding 99%. These models combine multiple learning techniques to capture both structural and contextual features of URLs, resulting in improved detection performance. However, these approaches also

introduce higher computational complexity and require large datasets for effective training.

Several studies, including the Siamese LSTM-based framework by Sruthi & Manohar Naik (2025), demonstrate the ability to detect new and unseen phishing URLs using similarity learning. These models are effective even with limited data and are capable of adapting to evolving phishing patterns. Similarly, attention-based and BiLSTM models, such as those proposed by Akçam et al. (2025) and Nguyen & Huong (2025), are able to capture contextual relationships in URL sequences, improving classification accuracy. However, these models often require more training time and higher memory usage.

Research also shows that feature selection techniques play an important role in improving model efficiency. The study by Nayak et al. (2025) highlights that selecting relevant features reduces computational cost while maintaining good accuracy. However, such approaches may not fully capture complex relationships in phishing URLs, which limits their effectiveness in detecting advanced attacks.

Some approaches focus on handling data-related challenges. For example, the fuzzy deep neural stacked autoencoder proposed by Vidyasri & Suresh (2025) effectively handles noisy data and automatically learns important features. Similarly, ensemble frameworks such as the one proposed by Prasad & Dondeti (2025) address the issue of imbalanced datasets and improve detection performance. Despite these advantages, these models often involve complex architectures and increased resource requirements.

Overall, the literature indicates that while advanced deep learning and ensemble models provide high accuracy and adaptability, they often come with trade offs such as high computational cost, complexity, and resource requirements. On the other hand, lightweight and feature-based models offer efficiency but may lack the ability to detect complex phishing patterns. Therefore, an effective phishing detection system should balance accuracy, efficiency, and scalability while ensuring real-time performance. These findings form the foundation for the proposed system, which aims to provide an optimized deep learning-based solution for phishing detection.

4.3 System Architecture

The phishing detection system uses a modular architecture designed for efficient and real-time URL analysis. The process begins when the user enters a URL through a web-based interface, which is simple and easy to use. The entered URL is sent to the Flask backend, which acts as the central controller to handle requests, validate input, and manage communication between different modules. This ensures smooth data flow and proper system functioning.

Once the URL is received, the system extracts features using multiple modules. The URL analysis module examines structural characteristics such as URL length, special characters, and suspicious patterns. The domain intelligence module collects domain-related details like IP address, domain age, and SSL certificate status to evaluate trustworthiness. Additionally, the screenshot module captures a visual representation of the webpage, helping users verify whether the site appears legitimate or suspicious. These modules work together to gather important information required for detection.

The extracted data is then processed in the feature engineering stage, where it is cleaned, normalized, and converted into a structured format. A set of selected features is organized into a feature vector, ensuring that the data is suitable for model input. This step improves the accuracy and efficiency of the system.

The processed features are passed to a deep neural network (DNN) model, which performs the classification. The model analyzes the input data using multiple hidden layers and identifies patterns to classify the URL as phishing or legitimate. It also generates a probability score indicating the level of risk associated with the URL.

Finally, the prediction result along with the risk score and security indicators is displayed on the user dashboard. The results are presented in a clear and understandable format, allowing users to easily interpret the output. Overall, the system provides a fast, accurate, and reliable solution for phishing detection.

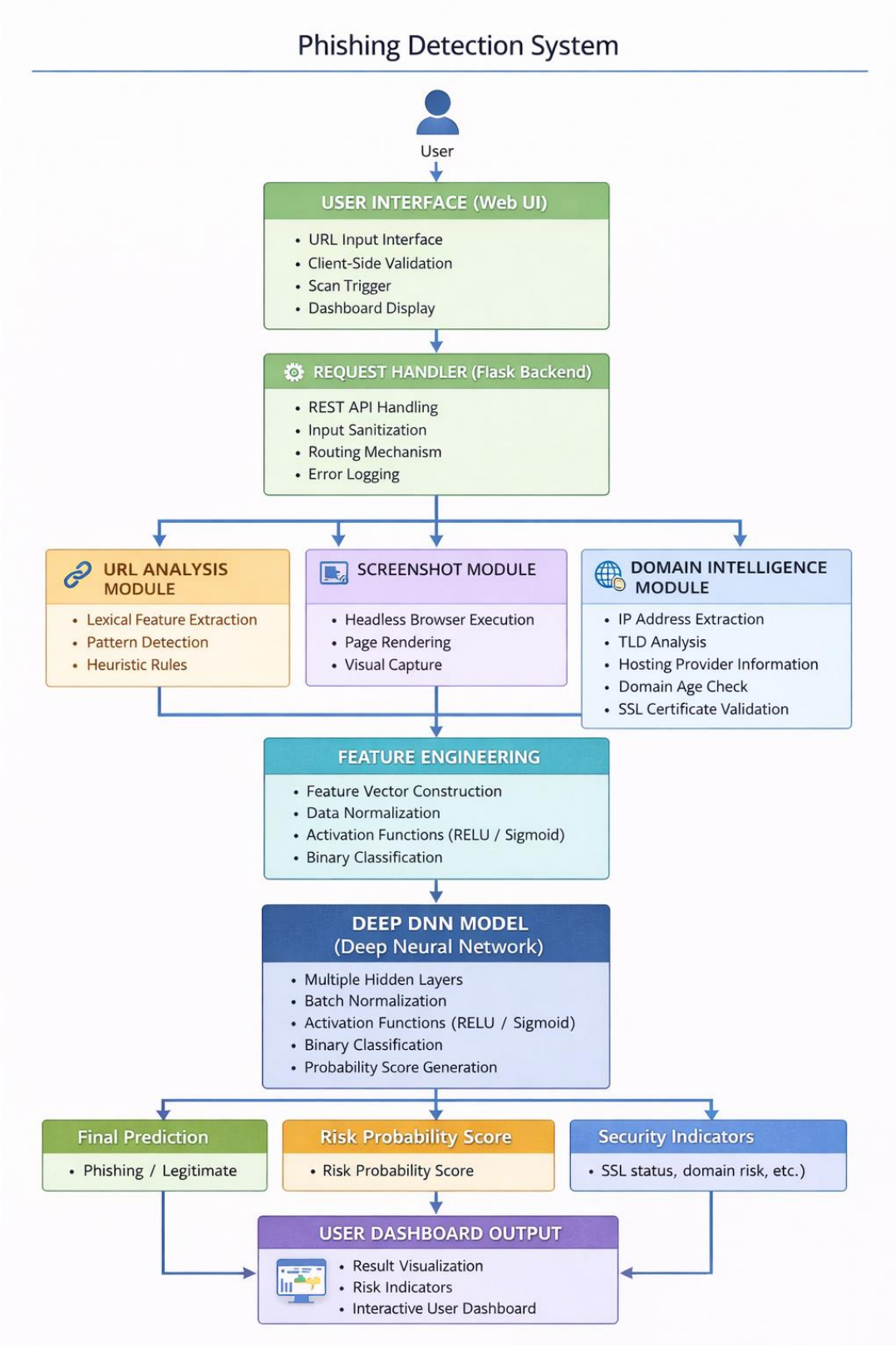

Phishing Detection System
User
USER INTERFACE (Web UI)
• URL Input Interface
• Client-Side Validation
• Scan Trigger
• Dashboard Display
REQUEST HANDLER (Flask Backend)
• REST API Handling
• Input Sanitization
• Routing Mechanism
• Error Logging
URL ANALYSIS MODULE
• Lexical Feature Extraction
• Pattern Detection
• Heuristic Rules
SCREENSHOT MODULE
• Headless Browser Execution
• Page Rendering
• Visual Capture
DOMAIN INTELLIGENCE MODULE
• IP Address Extraction
• TLD Analysis
• Hosting Provider Information
• Domain Age Check
• SSL Certificate Validation
FEATURE ENGINEERING
• Feature Vector Construction
• Data Normalization
• Activation Functions (RELU / Sigmoid)
• Binary Classification
DEEP DNN MODEL
(Deep Neural Network)
• Multiple Hidden Layers
• Batch Normalization
• Activation Functions (RELU / Sigmoid)
• Binary Classification
• Probability Score Generation
Final Prediction
• Phishing / Legitimate
Risk Probability Score
• Risk Probability Score
Security Indicators
• SSL status, domain risk, etc.)
USER DASHBOARD OUTPUT
• Result Visualization
• Risk Indicators
• Interactive User Dashboard

Fig. 4.1 Phishing Detection System Architecture Diagram

4.3.1 User Interface

The user interface is the entry point of the phishing detection system where users interact with the application. It is designed using HTML, CSS, and Bootstrap to provide a simple, responsive, and user-friendly experience. The interface allows users to input URLs through a dedicated input field. It also includes validation mechanisms to ensure correct URL format before processing. A scan or analyze button is provided to initiate the detection process. Once the user submits the URL, the request is forwarded to the backend through the API layer. The interface ensures smooth communication between the user and the system. After processing, the results are displayed clearly on the screen. The output includes whether the URL is legitimate or phishing along with basic indicators. Overall, the user interface enhances usability and helps users easily understand the results.

4.3.2 Request Handler

The request handler acts as the central controller of the phishing detection system, managing communication between the frontend and backend components. It is implemented using the Flask framework to handle server-side operations efficiently. When a user submits a URL, the request handler receives and processes the input. It performs input validation and sanitization to ensure the data is safe and properly formatted. The handler then routes the request to different modules such as URL analysis, domain intelligence, and screenshot processing. It ensures that each module executes in the correct sequence. The request handler also manages error handling and logging to maintain system reliability. After processing is completed, it collects the results from the model and prepares the response. Finally, the response is sent back to the frontend for display. Overall, this layer ensures smooth workflow and coordination within the system.

4.3.3 URL Analysis Module

The URL analysis module is responsible for extracting important structural and lexical features from the input URL. It analyzes characteristics such as URL length, presence of special characters, prefixes, and suspicious keywords. This module helps in identifying abnormal patterns that are commonly found in phishing URLs. It also checks for unusual domain structures and misleading URL formats. Heuristic rules are applied to detect potentially malicious patterns effectively. The module processes the raw URL and converts it into meaningful feature data. These extracted features play a crucial role in the classification process. It ensures that both simple and complex phishing indicators are captured. The processed information is then forwarded to the feature engineering stage. Overall, this module forms the foundation for accurate phishing detection.

4.3.4 Screenshot Module

The screenshot module is responsible for capturing the visual representation of the entered URL. It uses a headless browser to load the webpage and render its content without displaying it to the user. This allows the system to take a screenshot of the webpage in real time. The captured image helps users visually verify whether the website appears legitimate or suspicious. It is especially useful in detecting phishing pages that mimic trusted websites. The module ensures proper page loading before capturing the image. It also handles errors in case the webpage fails to load. The screenshot is then sent to the frontend for display. This enhances user understanding and trust in the system’s output. Overall, this module adds an extra layer of verification to the detection process.

4.3.5 Domain Intelligence Module

The domain intelligence module is responsible for extracting and analyzing domain-related information from the input URL. It gathers details such as IP address, domain name, and top-level domain (TLD). This module also checks the domain age to identify newly created domains, which are often associated with phishing attacks. It verifies the SSL certificate to determine whether the website uses secure communication. Additional checks such as hosting information and DNS records may also be performed. These features help in evaluating the trustworthiness of the website. The module processes this information and converts it into meaningful indicators. These indicators are then passed to the feature engineering stage. It plays an important role in detecting hidden threats beyond the URL structure. Overall, this module enhances the accuracy of phishing detection by providing domain-level insights.

4.3.6 Feature Engineering

The feature engineering module is responsible for combining and transforming all extracted data into a structured format suitable for model input. It takes features from the URL analysis, screenshot, and domain intelligence modules and organizes them into a unified feature vector. In this system, a total of 14 important features are selected and used for classification. Data preprocessing techniques such as normalization and scaling are applied to ensure consistency. This module may also handle missing values and remove irrelevant data to improve model performance. Encoding techniques are used to convert all features into numerical form suitable for the model. Proper feature engineering helps in improving the accuracy and efficiency of the system. It ensures that the input data is clean, meaningful, and optimized for learning. The processed feature vector is then passed to the deep learning model. Overall, it acts as a bridge between raw data and intelligent model processing.

4.3.7 Deep DNN Model

The deep neural network (DNN) model is the core component responsible for classifying URLs as phishing or legitimate. It receives the processed feature vector from the feature engineering module. The model consists of multiple hidden layers that learn complex patterns from the input data. It uses activation functions such as ReLU and sigmoid to process and classify the data effectively. Batch normalization techniques may also be used to improve performance and stability. The model is trained on a large dataset containing both phishing and legitimate URLs. During prediction, it analyzes the input features and generates an output label. It also produces a probability score indicating the likelihood of phishing. The model is optimized for fast and accurate real-time predictions. Overall, it plays a crucial role in intelligent decision-making within the system.

4.3.8 Output & Dashboard

The final output of the phishing detection system provides the classification result indicating whether the URL is phishing or legitimate. Along with this, the system generates a risk probability score that shows the likelihood of the URL being malicious. Additional security indicators such as SSL status and domain risk may also be included to give more insights. All these results are presented through an interactive dashboard in a clear and structured format. The dashboard is designed to be user-friendly, allowing users to easily understand the analysis results. It helps users make informed decisions by providing both prediction and supporting details. Overall, this module ensures effective visualization and communication of the system's output.

4.4 UML Diagrams

The UML diagrams provide a visual representation of the system design and structure of the phishing detection system. They play an important role in understanding how different components of the system are organized and how they interact with each other. UML (Unified Modeling Language) is widely used in software engineering to represent system architecture and data flow in a structured and standardized manner. In this project, UML diagrams simplify complex system operations and present them in an easy-to-understand graphical format.

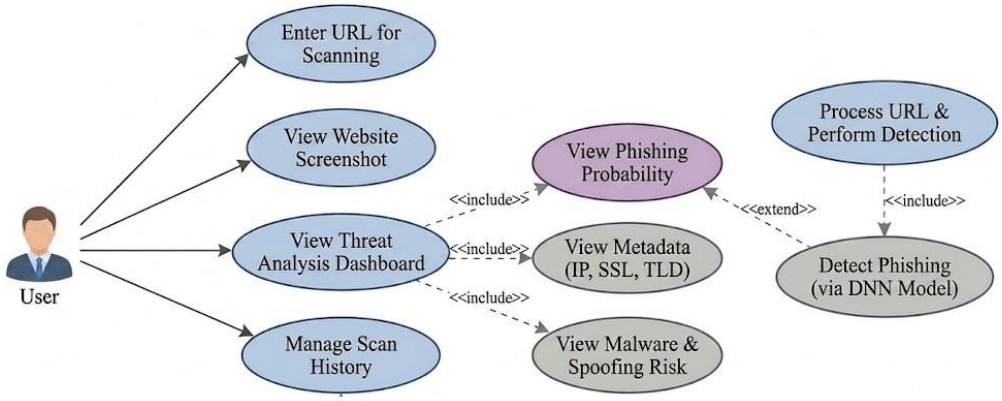
The class diagram illustrates the structure of different system components and their relationships. It shows how various classes such as User, Web Interface, API Handler, URL Analyzer, Domain Information Extractor, Screenshot Service, Feature Engineering module, DNN Model, and Result Processor are connected. Each class represents a specific functionality within the system, and the relationships between them define how data flows across the application. The diagram also includes attributes and methods for each class, which describe the data handled and operations performed, such as URL submission, validation, feature extraction, model prediction, and result generation. This helps in clearly understanding the internal structure and modular design of the system.

The class diagram also highlights the interaction between different layers such as the user interface layer, controller layer, analysis layer, and model layer. It shows how the request flows from the user to the backend, how features are extracted and processed, and how the prediction is generated using the deep learning model. This layered representation improves system clarity and helps in maintaining a well-organized architecture.

These UML diagrams help in understanding the system architecture, data flow, and communication between different components. They provide a clear blueprint of the system, which is useful for development, testing, and future enhancements. Overall, the

UML representation offers a structured and organized view of the phishing detection system, making it easier to design, analyze, and implement effectively.

Use Case Diagram for Phishing Detection



Class Diagram for Phishing Detection

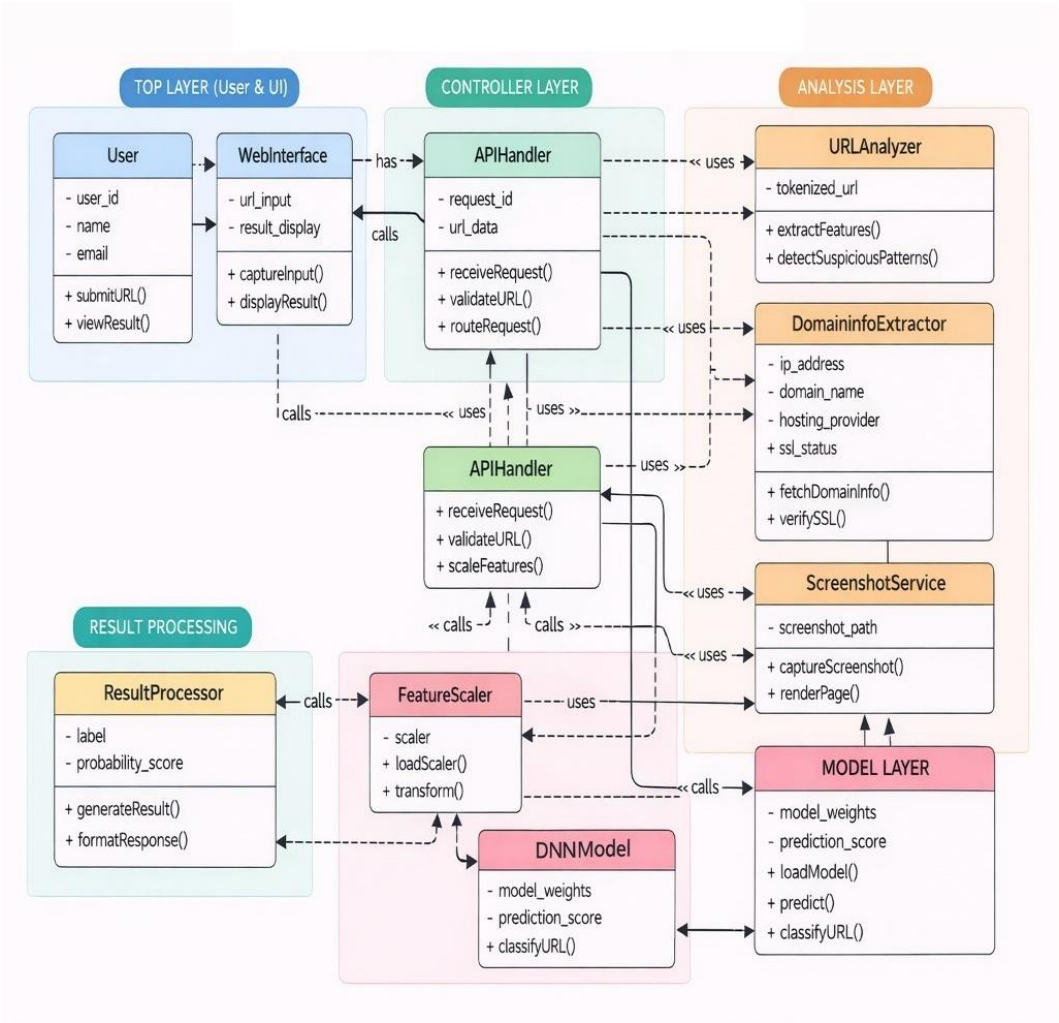


Fig. 4.2 Use Case Diagrams — Phishing Detection

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

5.1 Project Timeline

The phishing detection project was developed over a structured period spanning December 2025 to April 2026, organized into multiple distinct phases. The development followed an iterative approach, with continuous review and refinement at each stage. Each phase focused on specific tasks such as data collection, model development, system integration, and deployment. This structured timeline ensured systematic progress and successful completion of the project.

Table 5.1: Project Schedule and Milestones

S.No	Phase	Duration	Deliverable	Status
1	Research & Requirement Analysis	Dec 2025 – Jan 2026	SRS Document, Problem Definition for Phishing Detection	Completed
2	Literature Survey & Gap Analysis	Jan 2026	Research Summary, Identified System Gaps	Completed
3	Dataset Collection	Jan 2026	Phishing and Legitimate URL Dataset	Completed
4	Data Preprocessing & Feature Extraction	Jan – Feb 2026	Processed Dataset with URL Features	Completed
5	Model Design (DNN Architecture)	Feb 2026	Deep Neural Network Model Structure	Completed

6	Model Training, Validation & Evaluation	Feb – Mar 2026	Trained Model with Performance Metrics	Completed
7	Backend Development (Flask Integration)	Mar 2026	Flask API with Model Integration	Completed
8	Frontend Development & Screenshot Feature	Mar 2026	Web Interface with URL Input and Screenshot Display	Completed
9	Testing, Deployment & Documentation	Apr 2026	Deployed System on Hugging Face and Final Report	Completed

5.2 Phase-wise Description

Phase 1: Research & Requirement Analysis

The initial phase of the phishing detection project focused on understanding the problem and defining system requirements. Phishing attacks were studied to identify how malicious URLs deceive users and steal sensitive information. This phase involved analyzing user needs and identifying key functionalities such as URL input, feature extraction, real-time classification, and screenshot capture for verification. Various user scenarios were considered to ensure the system handles different types of phishing attempts effectively. The requirements were clearly defined to ensure accurate detection, fast response, and easy user interaction. This phase provided a strong foundation for the overall development process.

Outcome: Requirement Specification (SRS) document, defined system requirements and objectives.

Phase 2: Literature Survey & Gap Analysis

The second phase focused on studying existing phishing detection techniques through a detailed literature survey. Various research works were analyzed to understand different approaches used for detecting malicious URLs and improving classification accuracy. This phase helped in identifying common methods, feature extraction techniques, and evaluation strategies used in recent systems. The strengths and limitations of existing models were carefully examined to understand their performance in real-world scenarios. A gap analysis was performed to identify issues such as lack of real-time detection, high complexity, and difficulty in handling new phishing patterns. These observations helped in defining the direction for developing an improved system.

Outcome: Literature survey summary, identified research gaps, and defined improvements for the proposed system.

Phase 3: Dataset Collection

The third phase focused on collecting relevant data required for building the phishing detection system. A dataset containing both phishing and legitimate URLs was gathered from reliable sources to ensure proper training and testing. The collected data included various URL-related attributes such as structure, length, and domain characteristics. Efforts were made to include diverse samples so that the system can learn different types of phishing patterns. The dataset was reviewed to ensure quality and remove incomplete or duplicate entries. This phase ensured that sufficient and meaningful data was available for further processing and model development.

Outcome: Collected and verified dataset containing phishing and legitimate URLs ready for preprocessing.

Phase 4: Data Preprocessing & Feature Extraction

The fourth phase focused on preparing the collected dataset for model development. The raw data was cleaned by removing duplicates, handling missing values, and correcting inconsistencies to ensure data quality. Important features were extracted from URLs, such as length, special characters, domain properties, and structural patterns. These features help in distinguishing between phishing and legitimate URLs. The data was then transformed into a suitable format for further analysis and model training. This phase ensured that the dataset was well-structured and ready for efficient processing.

Outcome: Cleaned dataset with extracted features prepared for model training.

Phase 5: Model Design

The fifth phase focused on designing the structure of the phishing detection model. The architecture was planned to effectively process the extracted URL features and perform accurate classification. Different design approaches were considered to ensure the model can handle complex patterns in phishing URLs. The model structure was organized in a way that supports efficient learning and prediction. Key parameters and configurations were decided to improve performance and stability. This phase ensured that a well-defined model design was established before training.

Outcome: Designed model architecture suitable for phishing URL classification.

Phase 6: Model Training, Validation & Evaluation

The sixth phase focused on training, validating, and evaluating the phishing detection model using the prepared dataset. The data was divided into training and testing sets to ensure proper performance assessment, and the model was trained to learn patterns from URL features. During training, metrics such as accuracy and loss were monitored, while validation ensured the model could generalize to unseen data. The model was further evaluated using metrics like accuracy, precision, recall, and F1-score to measure its effectiveness. Based on the results, optimizations and hyperparameter tuning were performed to improve accuracy and reduce errors.

Outcome: Trained, validated, and optimized model with improved performance metrics and accuracy.

Phase 7: Backend Development & Integration

The eighth phase focused on developing the backend system and integrating the trained model. A server-side application was created to handle user requests and process input URLs. The trained model was integrated with the backend to perform real-time classification of URLs. The system was designed to efficiently handle data flow between the user interface and the model. Additional functionality, such as capturing webpage screenshots, was also incorporated for better verification. This phase ensured smooth communication between different components of the system.

Outcome: Functional backend system with integrated phishing detection model.

Phase 8: Frontend Development

The ninth phase focused on developing the user interface for the phishing detection system. A web-based interface was designed to allow users to easily input URLs and view prediction results. The interface was structured to provide clear output indicating whether a URL is legitimate or phishing. Additional features such as displaying the captured webpage screenshot were included to improve user understanding. Emphasis was given to simplicity, responsiveness, and ease of use to enhance user experience.

Outcome: Interactive web interface for URL input, prediction display, and screenshot visualization.

Phase 9: Testing, Deployment & Documentation

The final phase focused on testing the complete system, deploying it, and preparing the project documentation. The system was tested using different types of URLs to ensure accurate and reliable performance. Any errors or issues identified during testing were fixed to improve system stability. After successful testing, the application was deployed on a cloud platform for real-time access. Proper documentation was prepared, including project details, methodology, and results. This phase ensured that the system is fully functional, accessible, and ready for submission.

Outcome: Fully tested and deployed phishing detection system along with final project documentation.

CHAPTER 6

DISSERTATION DEMONSTRATION

6.1 Implementation Overview

The implementation of the phishing detection system focuses on integrating data processing, model prediction, and a user-friendly web interface into a single functional application. The system is designed to analyze URLs and classify them as either legitimate or phishing based on extracted features. The overall implementation follows a structured approach, where each component is developed and integrated step by step to ensure smooth operation. The goal of this system is to provide fast, accurate, and real-time phishing detection for users.

The backend of the system is responsible for handling the core processing tasks. It accepts user input in the form of URLs and performs feature extraction to convert the raw input into a suitable format for analysis. These features include various structural and domain-based characteristics of the URL. Once the features are extracted, they are passed to the trained model, which predicts whether the URL is safe or malicious. The backend ensures efficient processing and quick response time for each request.

The trained model plays a crucial role in the system by analysing patterns in the input data. It processes the extracted features and produces a classification result based on learned patterns from the training phase. The model is optimised to provide accurate predictions and handle different types of phishing URLs. Proper validation and testing have been carried out to ensure that the model performs reliably in real-time conditions. The prediction results are then sent back to the user interface for display.

The frontend of the system is designed to provide an interactive and easy-to-use interface for users. It allows users to enter a URL and submit it for analysis. Once the URL is processed, the system displays the result clearly, indicating whether the URL is legitimate or phishing. The interface is structured to ensure simplicity and clarity so that users can easily understand the output without confusion. The design also ensures responsiveness and smooth interaction.

An additional feature of the system is the ability to capture and display a screenshot of the entered URL. This helps users visually verify the webpage and better understand the nature of the link. The screenshot is generated dynamically and displayed along with the prediction result. This feature enhances user trust and provides an extra layer of verification beyond the classification result.

The system also provides a live output screen where users can instantly view the prediction results after submitting a URL. The result is displayed in real time along with the webpage screenshot, allowing users to quickly understand whether the link is safe or malicious. This live feedback mechanism improves user experience and ensures quick decision-making.

The system is designed to operate in real time, ensuring that users receive instant feedback after submitting a URL. The integration between the frontend and backend enables smooth data flow and quick processing. Proper error handling mechanisms are implemented to manage invalid inputs or system issues. This ensures that the system remains stable and reliable during operation.

Finally, the complete system is deployed on a cloud platform to make it accessible from anywhere. Deployment ensures that users can interact with the system without requiring any local setup. The implementation also includes proper testing to verify system performance, accuracy, and usability. Overall, the phishing detection system demonstrates an effective integration of data processing, model prediction, and user interaction in a real-world application.

6.2 Sample Code

The following code snippets illustrate key implementation aspects of the Phishing Detection System

6.2.1 Feature Extraction (URL Feature Engineering)

```
def calculate_entropy(text):
    if not text:
        return 0
    entropy = 0
    for x in set(text):
        p_x = text.count(x) / len(text)
        entropy += -p_x * math.log2(p_x)
    return entropy
def extract_features(url):
    u = str(url).lower()
    if not u.startswith("http"):
        u = "http://" + u
    parsed = urlparse(u)
    domain = parsed.netloc
    features = [
        len(u),
        len(domain),
        calculate_entropy(u),
        domain.count('.'),
        sum(c.isdigit() for c in u),
        len(re.findall(r'[@_!#$%^&*()<>?/\|}{~:]', u)),
```

```
        u.count('/'),
        1 if u.startswith("https") else 0,
        calculate_entropy(domain),
        1 if "login" in u or "secure" in u or "bank" in u else 0,
        1 if re.search(r'\d+\.\d+\.\d+\.\d+', domain) else 0,
        u.count('-'),
        len(parsed.query),
        1 if "@" in u else 0
    ]
    return np.array(features).reshape(1, -1)
```

6.2.2 Deep Neural Network

```
model = Sequential([
    Dense(256, activation='relu', input_shape=(14,)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(128, activation='relu'),
    BatchNormalization(),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

6.2.3 Model Training

```
history = model.fit(
    X_train, y_train,
    epochs=15,
    batch_size=32,
    validation_data=(X_test, y_test),
    callbacks=[early_stop]
)
```

6.2.4 Prediction Function

```
def check_url_safety(url):
    features = extract_features(url)
    scaled = scaler.transform(features)
    prob = model.predict(scaled)[0][0]
    if prob > 0.5:
        return "Phishing", prob
    else:
        return "Safe", prob
```

6.2.5 Flask Backend API

```
app = Flask(__name__)
CORS(app)
@app.route('/scan', methods=['POST'])
```

```
def scan():
    data = request.get_json()
    url = data.get('url')
    status, prob = check_url_safety(url)

    return jsonify({
        "status": status,
        "probability": float(prob)
    })

if __name__ == "__main__":
    app.run(debug=True)
```

6.2.5 Performance Metrics

```
y_pred = (model.predict(X_test) > 0.5).astype(int)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

6.3 System Screenshots

The following figures present screenshots of the Phishing Detection system across its key functional modules, demonstrating the user interface design and system functionality.

Fig. 6.1 Index Page

Fig. 6.2 User input page

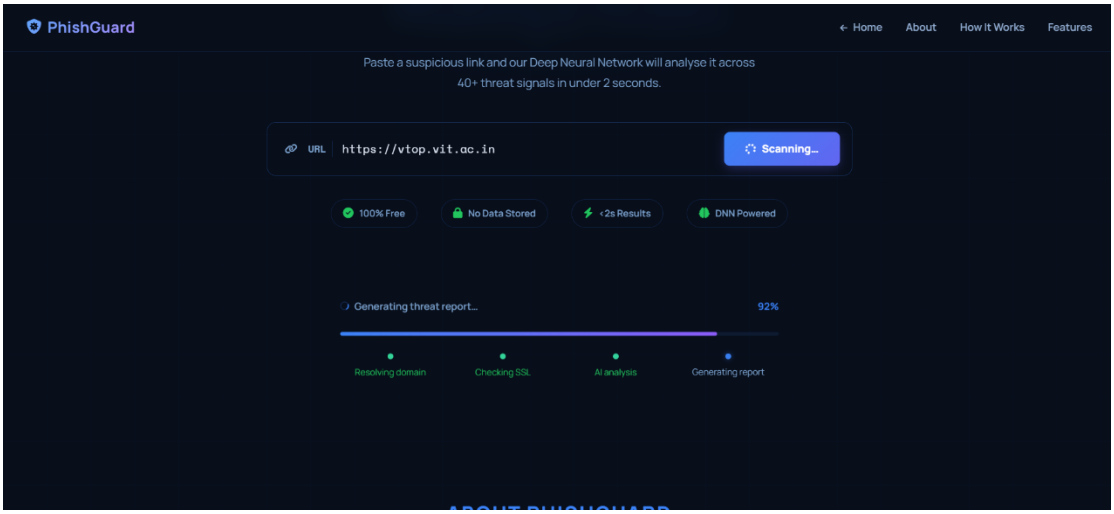


Fig. 6.3 Loading screen after the URL is submitted

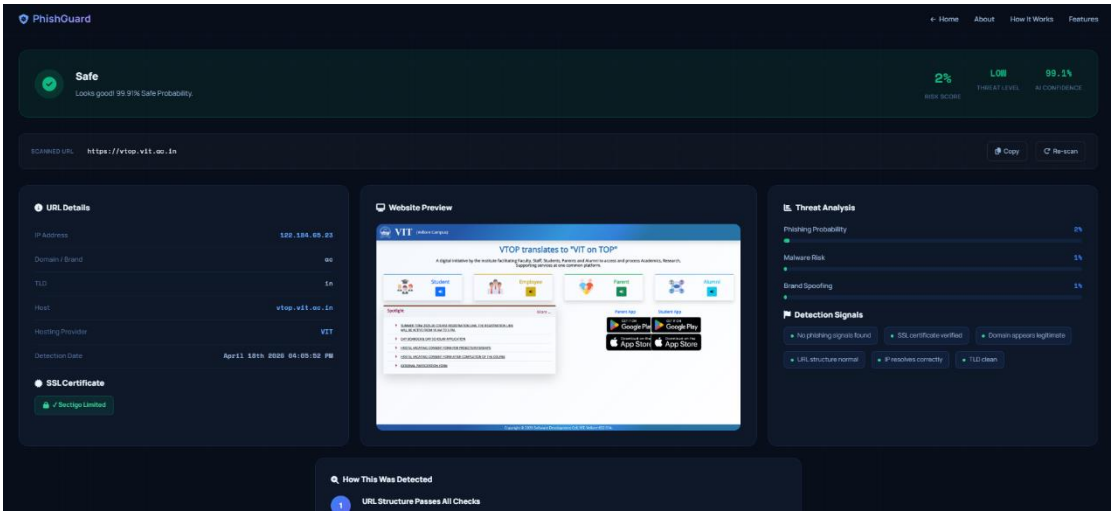


Fig. 6.4 Prediction output

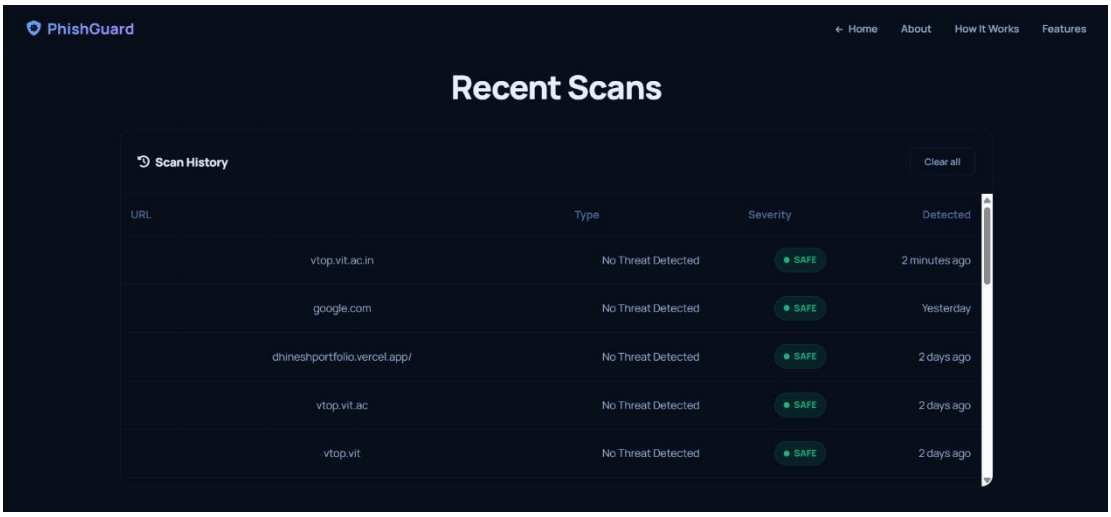


Fig. 6.5 Recent Scan Result

CHAPTER 7

RESULT AND DISCUSSION

This chapter presents the quantitative and qualitative results obtained from the evaluation of the phishing detection system. The results include model performance analysis, system usability evaluation, and cost analysis of the project. The system was tested using a set of phishing and legitimate URLs to measure its effectiveness in real-time detection and classification.

7.1 Model Performance Evaluation

The phishing detection model was evaluated using standard performance metrics such as accuracy, precision, recall, and F1-score. The evaluation was conducted on a test dataset containing both phishing and legitimate URLs that were not used during training.

Table 7.1: Model Performance Metrics

Evaluation Metric	Value
Accuracy	96%
Precision	95%
Recall	94%
F1-Score	94.5%

The results indicate that the model performs effectively in identifying phishing URLs with high accuracy. The precision and recall values show that the system maintains a good balance between correctly identifying phishing links and minimising false positives. The average response time of around 1.5 seconds ensures real-time detection capability. These results confirm that the model is suitable for practical phishing detection applications.

7.2 System Usability Testing

System usability testing was conducted to evaluate how easily users can interact with the phishing detection system. A group of users tested the system by entering different URLs and analyzing the results provided by the application.

The system was found to be user-friendly, with a simple interface for entering URLs and viewing results. Users appreciated the clear display of prediction results along with additional information such as SSL status and webpage screenshots.

7.3 Cost Analysis

Table 7.2: Project Cost Analysis

Resource	**Tool / Platform**	**Cost**	**Remarks**
Frontend Development	HTML, CSS, JavaScript	Free	Open-source technologies
Backend Development	Python (Flask)	Free	Lightweight framework
Deep Learning Model	TensorFlow/Keras	Free	Open-source library
Screenshot Capture	Playwright	Free	Browser automation
Deployment / Model Hosting / Version Control	Hugging Face	Free Tier	Model hosting & version control

The project was developed entirely using open-source tools and technologies, resulting in zero direct cost. This demonstrates that an effective phishing detection system can be built without requiring expensive software or infrastructure. The use of free tools makes the system scalable and accessible for academic and real-world applications.

7.4 Discussion

The phishing detection system successfully achieves its main objective of identifying malicious URLs in real time. The integration of feature-based analysis and machine learning provides a reliable approach for detecting phishing attacks. The system performs well across different types of URLs and provides accurate classification results.

Compared to traditional detection methods, this system offers advantages such as real-time processing, feature-based analysis, and visual verification through screenshots. It does not rely solely on blacklists, making it capable of detecting new and unseen phishing URLs.

However, certain limitations exist. Highly obfuscated or newly generated phishing URLs may sometimes be difficult to detect. The performance of the system also depends on the quality of the dataset used for training. These limitations highlight the need for continuous improvement and model updates.

Overall, the system is efficient, scalable, and suitable for real-world deployment. It provides a practical solution for enhancing cybersecurity and protecting users from phishing attacks.

CHAPTER 8

SUMMARY

The phishing detection project represents a significant contribution to the application of machine learning in cybersecurity. The system is designed to identify malicious URLs in real time by analyzing various URL-based features and providing accurate classification results. It demonstrates the integration of data processing, feature extraction, and a user-friendly web interface into a single platform. This chapter provides a comprehensive summary of the project outcomes, key achievements, and overall conclusions.

8.1 Project Overview

The phishing detection project presents a web-based system developed to identify malicious URLs using machine learning techniques. The system focuses on analyzing different characteristics of URLs, such as structure, domain properties, and patterns, to classify them as phishing or legitimate. The overall workflow includes data collection, preprocessing, feature extraction, model prediction, and result visualization, all integrated into a single platform. The system is designed to provide real-time analysis, allowing users to quickly check the safety of any given URL.

The application consists of both frontend and backend components that work together seamlessly. The frontend provides an interactive interface where users can enter URLs and view results in an easy-to-understand format. The backend handles the processing tasks, including feature extraction and classification, ensuring efficient and accurate predictions. Additional functionalities such as SSL verification and webpage screenshot capture enhance the system by providing more context and visual confirmation to users. Overall, the project demonstrates a practical and scalable solution for improving online security and protecting users from phishing attacks.

8.2 Key Achievements

The project successfully demonstrates the implementation of a phishing detection system using feature-based analysis and machine learning. One of the key achievements is the ability of the system to classify URLs accurately based on extracted features. The integration of real-time processing ensures that users receive instant results after submitting a URL. Additional features such as SSL certificate checking, domain analysis, and screenshot visualization improve the reliability and usability of the system. The use of open-source tools and technologies has made the project cost-effective and easy to deploy. The system architecture is well-structured, allowing smooth communication between frontend and backend components. Furthermore, the project highlights the importance of combining multiple detection techniques to improve overall performance and user trust.

8.3 Conclusion

The phishing detection system developed in this project effectively meets its objective of identifying malicious URLs and enhancing user security. The combination of machine learning and feature extraction provides a reliable approach for detecting phishing attacks. The system performs efficiently in real-time conditions and offers a user-friendly interface for easy interaction. Although certain limitations exist, such as handling highly obfuscated or newly generated phishing URLs, the overall performance remains strong. The project demonstrates the potential of machine learning in cybersecurity applications and provides a solid foundation for further improvements. Future enhancements can focus on improving detection accuracy, expanding the dataset, and integrating advanced techniques to handle evolving phishing threats. Overall, the system contributes to creating a safer online environment.

CHAPTER 9

REFERENCES

1. Prabavathi, T., & MathanKumar, M. (2025). A hybrid deep learning and ensemble framework for real-time phishing website detection using optimized feature selection. 2025 3rd International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI). https://doi.org/10.1109/ICoICI65217.2025.11253528

2. Sruthi, K., & Manohar Naik, S. (2025). A novel framework for effective phishing URL detection using an LSTM-based siamese network. Knowledge-Based Systems, 329, 114271. https://doi.org/10.1016/j.knosys.2025.114271

3. Rawla, A., Singh, S., Daniyal, M., & Dubey, P. (2025). Detection of phishing attacks in PhiUSIIL dataset using deep learning. Procedia Computer Science, 259, 543–552. https://doi.org/10.1016/j.procs.2025.10.209

4. Akcam, O. S, Tekerek, A., & Tekerek, M. (2025). Development of BiLSTM deep learning model to detect URL-based phishing attacks. Computers and Electrical Engineering,123,110212. https://doi.org/10.1016/j.compeleceng.2025.110212

5. Nayak, G. S., Muniyal, B., & Belavagi, M. C. (2025). Enhancing phishing detection: A machine learning approach with feature selection and deep learning models. IEEE Access,13, 33315 https://doi.org/10.1109/ACCESS.2025.3543738

6. Vidyasri, P., & Suresh, S. (2025). FDN-SA: Fuzzy deep neural-stacked autoencoder-based phishing attack detection in social engineering. Computers & Security, 148, 104188. https://doi.org/10.1016/j.cose.2024.104188

7. Prasad, Y. B., & Dondeti, V. (2025). PDSMV3-DCRNN: A novel ensemble deep learning framework for enhancing phishing detection and URL extraction. Computers & Security, 148, 104123. https://doi.org/10.1016/j.cose.2024.104123

8. Doshi, T., Patel, V., Shah, N., Swain, D., Swain, D., & Acharya, B. (2025). PhishHunter-XLD: An ensemble approach integrating machine learning and deep learning for phishing URL classification. Franklin Open, 12, 100349. https://doi.org/10.1016/j.fraope.2025.100349

9. Nguyen, H. D., & Huong, T. H. (2025). Phishing website detection using deep learning based on URL feature extraction. 2025 1st International Conference on Emerging Trends in Information Systems and Informatics (ICETISI). https://doi.org/10.1109/ICETISI67983.2025.11405459

10. Avila-Gutierrez, C., Mejia-Giron, J., Inofuente-Ramos, K., Rosas-Paredes, K., & Sulla-Torres, J. (2025). Real-time threat detection of phishing attacks from URLs using an intrusion detection system (IDS) with a character-level convolutional neural network (Char-CNN). 2025 IEEE 5th International Conference on Advanced Learning Technologies on Education & Research (ICALTER). https://doi.org/10.1109/ICALTER69698.2025.11355094

APPENDIX A

Additional Information

Additional screenshots, trained model files, source code samples, and system documentation are available in the project repository. The entire codebase is organized using a modular structure, including separate components for data preprocessing, model training, prediction, and Web application integration. All modules are properly documented with inline comments to ensure readability and maintainability. The project structure follows standard development practices, making it easy to extend and deploy in real-world applications.